Name: Irisha Deshpande	Class: T1	Roll No: 2401022
Batch: T11	Subject: Agile Software Development & DevOps	Deptartment: AI & DS

Aim: To perform version control operations using Git.

This experiment maps to LO2.

LO2: Introduction to Git, GitHub, Basic operations: clone, commit, push, pull, Branching strategies and merging, Git workflows: Feature branching, Git Flow, Handling merge conflicts, GitHub Actions basics

Git Commands — Version Control Experiment

Setup & Configuration

1. git --version — Checks which version of Git is installed on the system.
2. git config --global user.name “ahsiri14" — Sets the username attached to your commits.
3. git config --global user.email “irishaa1404@gmail.com” — Sets the email attached to your commits.

```
dquote>
(base) irishadeshpande@Irishas-MacBook-Air ~ % git config --global user.name "ahsiri14"
(base) irishadeshpande@Irishas-MacBook-Air ~ % git config --global user.email "jacynthia389@gmail.com"
(base) irishadeshpande@Irishas-MacBook-Air ~ % git --version
git version 2.39.5 (Apple Git-154)
```

Creating the Repository

4. mkdir git_experiment — Creates a new folder for the project.
5. cd git_experiment — Moves into that folder.
6. git init — Initializes a new Git repository in the current folder.
7. git status — Shows the current state of the working directory (tracked/untracked/staged files).

```
(base) irishadeshpande@Irishas-MacBook-Air ~ % mkdir git_experiment
(base) irishadeshpande@Irishas-MacBook-Air ~ % cd git_experiment
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git init
Initialized empty Git repository in /Users/irishadeshpande/git_experiment/.git/
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
(base) irishadeshpande@Irishas-MacBook-Air git_experiment %
```

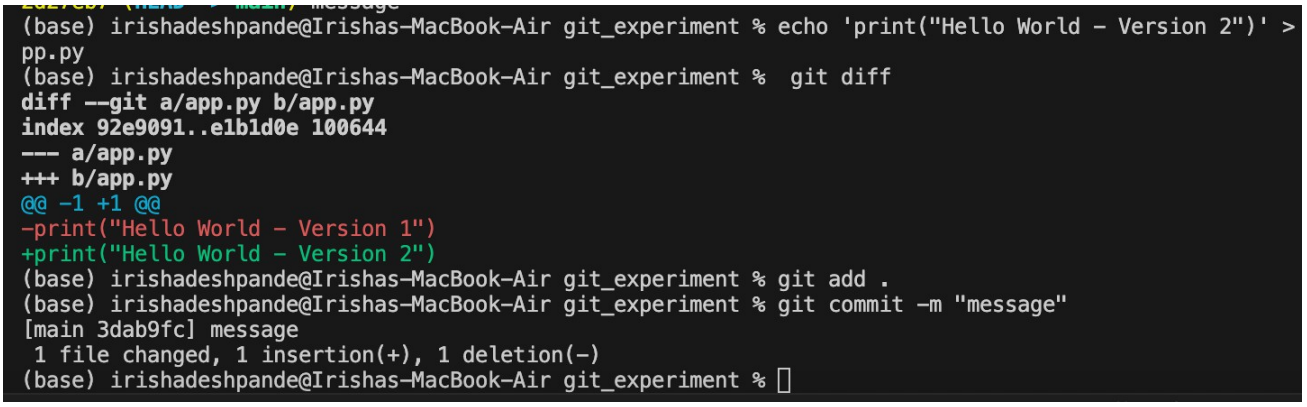
Creating & Committing Files

8. echo 'print("Hello World - Version 1")' > app.py — Creates a Python file with starter code.
9. git add app.py — Stages a specific file, marking it ready to be committed.
10. git commit -m "message" — Saves a permanent snapshot of staged changes with a message.
11. git log --oneline — Displays commit history in a short, one-line-per-commit format.



Editing & Comparing Versions

12. echo 'print("Hello World - Version 2")' > app.py — Edits the file to create a new version.
13. git diff — Shows line-by-line differences between the working file and the last commit.
14. git add . — Stages all changed/new files in the folder at once.
15. git commit -m "message" — Commits the newly staged changes.



Branching

16. git branch — Lists all branches and shows which one is currently active.
17. git branch feature-branch — Creates a new branch without switching to it.
18. git checkout feature-branch — Switches the working directory to the specified branch.
19. echo 'print(...)' > app.py — Edits the file while on the new branch.
20. git commit -am "message" — Stages tracked files and commits in a single step.
21. git checkout master — Switches back to the main branch.
22. git merge feature-branch — Combines changes from feature-branch into the current branch.
23. git checkout -b test-branch — Creates and switches to a new branch in one command.

```
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git branch
* main
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git branch-feature branch
git: 'branch-feature' is not a git command. See 'git --help'.
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git branch feature-branch
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git checkout feature branch
error: pathspec 'feature' did not match any file(s) known to git
error: pathspec 'branch' did not match any file(s) known to git
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git checkout feature-branch
Switched to branch 'feature-branch'
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % echo 'print(...)' > app.py
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git commit -am "message"
[feature-branch fa73375] message
 1 file changed, 1 insertion(+), 1 deletion(-)
(base) irishadeshpande@Irishas-MacBook-Air git_experiment %  git checkout master
error: pathspec 'master' did not match any file(s) known to git
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git checkout main
Switched to branch 'main'
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git merge feature-branch
Updating 3dab9fc..fa73375
Fast-forward
 app.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git checkout -b test-branch
Switched to a new branch 'test-branch'
```

Undoing Changes / Rollback

24. git restore app.py — Discards uncommitted changes in a file, reverting it to the last commit.
25. git add app.py — Stages a file (used before demonstrating unstage below).
26. git restore --staged app.py — Unstages a file without deleting the actual edit.
27. git reset --soft HEAD~1 — Undoes the last commit but keeps changes staged.
28. git reset --hard HEAD~1 — Completely undoes the last commit and discards the changes (destructive).
29. git revert <commit-id> — Creates a new commit that undoes an older commit's changes, without deleting history (the safe rollback method).

```
(base) irishadeshpande@Irishas-MacBook-Air git_experiment %  git restore app.py
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git add app.py
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git restore --staged app.py
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git reset --soft HEAD~1
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git reset --hard HEAD~1
HEAD is now at 2d27eb7 message
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git revert <commit-id>
zsh: parse error near `\n'
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git revert 2d27eb7
[test-branch 7ae0bb7] Revert "message"
 1 file changed, 1 deletion(-)
 delete mode 100644 app.py
(base) irishadeshpande@Irishas-MacBook-Air git_experiment %
```

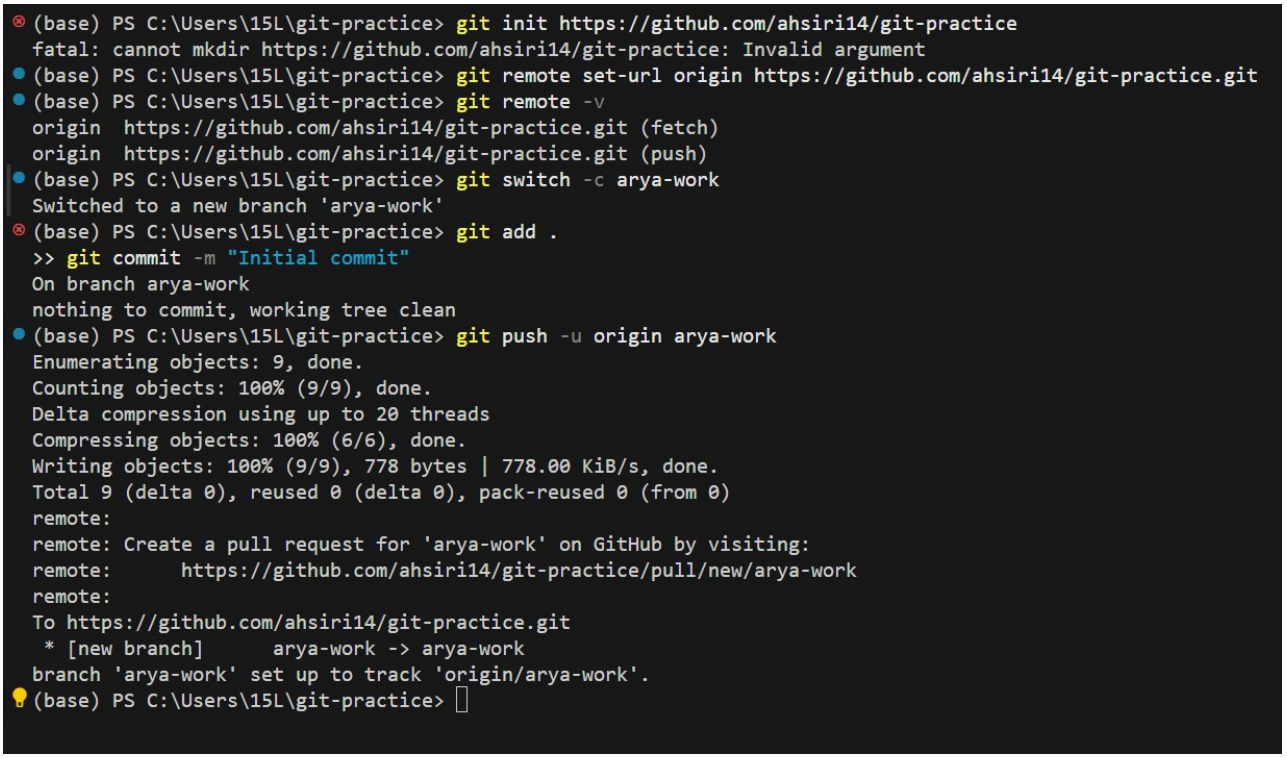
Viewing Final State

30. git log --oneline — Final check showing the complete commit history after all operations.

```
delete mode 100644 app.py
(base) irishadeshpande@Irishas-MacBook-Air git_experiment % git log --oneline
7ae0bb7 (HEAD -> test-branch) Revert "message"
2d27eb7 message
```

Collaboration

31. git remote set-url origin https://github.com/ahsiri14/git-practice -Changes the URL of the existing remote repository named origin
32. git remote -v - Displays the remote repositories configured for fetch and push operations.
33. git-practice (fetch)- Shows the remote URL Git uses to fetch changes from the repository.
34. git-practice (push)- Shows the remote URL Git uses to push changes to the repository.
35. git switch -c arya-work- Creates a new branch named and arya-work switches to it.
36. git push -u origin work- Pushes the arya-work branch to the remote repository and sets it as the default upstream branch.



Conclusion:This Git experiment demonstrated the basic version control workflow, including repository creation, file tracking, committing changes, branching, merging, reverting changes, and managing remote repositories. By performing these operations, we gained practical experience in using Git to track project history, collaborate efficiently, and safely manage different versions of code.